CS 213，秋季 2001 年 动态内存分配实验：编写动态存储分配器 分配：11 月 2 日星期五，截止：11 月 20 日星期二晚上 11:59

Cory Williams (cgw@andrew.cmu.edu) 是这项作业的主要负责人。

1 引言

在这个实验中，你将编写一个用于 C 程序的动态存储分配器，即你自己的 malloc、free 和 realloc 例程版本。我们鼓励你富有创意地探索设计空间，实现一个正确、高效且快速的分配器。

2 物流

你可以组成最多两人的小组。任何关于作业的解释和修订都将发布在课程网页上。

3 分发说明

特定站点说明：在此处插入一段解释学生应如何下载 malloclab-handout.tar 文件的段落。

首先将 malloclab-handout.tar 复制到计划进行工作的受保护目录中。然后输入命令：tar xvf malloclab-handout.tar。这将导致多个文件被解压到该目录中。您将修改并提交的只有 mm.c 文件。mdriver.c 程序是一个驱动程序，允许您评估您的解决方案的性能。使用 make 命令生成驱动代码，并用./mdriver -V 命令运行它。（-V 标志显示有用的摘要信息。）
查看 mm.c 文件时，您会发现一个 C 结构 team，您应该在其中插入关于您编程团队的一个或两个成员的请求识别信息。立即这样做，以免忘记。

当您完成实验后，您只需提交一个文件（mm.c），其中包含您的解决方案。

4 如何进行实验

你的动态存储分配器将包含以下四个函数，这些函数在 mm.h 中声明并在 mm.c 中定义。

```
int mm_init(void); void *mm_malloc(size_t size); void
mm_free(void *ptr); void *mm_realloc(void *ptr, size_t
size);
```

我们提供的 mm.c 文件实现了我们认为最简单但功能上仍然正确的 malloc 包。以这个为基础，修改这些函数（可能还需要定义其他私有静态函数），使它们遵循以下语义：

- mm_init: 在调用 mm_malloc、mm_realloc 或 mm_free 之前，应用程序（即你将用来评估你的实现的跟踪驱动驱动程序）会调用 mm_init 来执行任何必要的初始化，例如分配初始堆区域。如果初始化过程中出现问题，返回值应为-1，否则为 0。

- mm_malloc: mm_malloc 例程返回一个指向至少为 size 字节的已分配块有效负载的指针。整个已分配的块应位于堆区域内，并且不应与其他任何已分配的块重叠。
  我们将你的实现与标准 C 库（libc）中提供的 malloc 版本进行比较。由于 libc malloc 始终返回对齐到 8 字节的负载指针，你的 malloc 实现也应如此，并始终返回 8 字节对齐的指针。

- mm_free: mm_free 例程释放 ptr 指向的块。它不返回任何内容。该例程仅保证在传递的指针（ptr）是由先前的 mm_malloc 或 mm_realloc 调用返回且尚未被释放时才能正常工作。

- mm_realloc：mm_realloc 例程返回一个指向至少为 size 字节的已分配区域的指针，并具有以下约束。

  – 如果 ptr 为 NULL，调用等效于 mm_malloc(size)；– 如果 size 等于零，调用等效于 mm_free(ptr);

  – 如果 ptr 不为 NULL，它必须是由之前的 mm_malloc 或 mm_realloc 调用返回的。
    对 mm_realloc 的调用会改变 ptr 指向的内存块的大小（旧块）为 size 字节，并返回新块的地址。请注意，新块的地址可能与旧块相同，也可能不同，具体取决于您的实现、旧块中的内部碎片量以及 realloc 请求的大小。
    新块的内容与旧 ptr 块相同，直到旧大小和新大小的最小值。其余部分都是未初始化的。例如，如果旧块是 8 字节，新块是 12 字节，那么新块的前 8 字节与旧块的前 8 字节相同。

旧块的字节和最后 4 个字节是未初始化的。类似地，如果旧块是 8 个字节而新块是 4 个字节，那么新块的内容与旧块的前 4 个字节相同。

这些语义与相应的 libc malloc、realloc 和 free 例程的语义相匹配。在 shell 中输入 man malloc 以获取完整文档。

5 堆一致性检查器

动态内存分配器在编程正确和高效方面以复杂著称。它们难以正确编程，因为涉及大量未类型化的指针操作。你会发现编写一个堆检查器非常有帮助，该检查器扫描堆并检查其一致性。

堆检查器可能检查的一些示例包括:

- 自由列表中的每个块是否都标记为自由?
- 是否存在任何连续的自由块通过某种方式逃过了合并?
- 每个空闲块是否真的在空闲列表中?
- 空闲列表中的指针是否指向有效的空闲块?
- 是否有任何已分配的块重叠?
- 堆块中的指针是否指向有效的堆地址?

你的堆检查器将包含在 mm.c 文件中的函数 int mm check(void)。它将检查你认为是谨慎的任何不变式或一致性条件。只有当你的堆一致时，它才会返回非零值。你不受所列建议的限制，也不必检查所有这些条件。当 mm check 失败时，鼓励你打印出错误消息。
这个一致性检查器用于你在开发过程中的调试。当你提交 mm.c 时，确保移除对 mm check 的所有调用，因为它们会降低你的吞吐量。对于 mm check 函数，会根据风格给予评分。确保添加注释并记录你正在检查的内容。

6 支持例程

memlib.c 包模拟了你的动态内存分配器的内存系统。你可以在 memlib.c 中调用以下函数:

- void *mem_sbrk(int incr): 通过 incr 字节扩展堆，其中 incr 是一个正的非零整数，并返回一个通用指针，指向新分配堆区域的第一字节。其语义与 Unix sbrk 函数相同，只是 mem_sbrk 仅接受一个正的非零整数参数。

- void *mem_heap_lo(void): 返回堆中第一个字节的通用指针。
- void *mem_heap_hi(void): 返回堆中最后一个字节的通用指针。
- size_t mem_heapsize(void): 返回堆当前的字节大小。
- size_t mem_pagesize(void): 返回系统的页大小（Linux 系统为 4K 字节）。

7 The Trace-driven Driver Program

malloclab-handout.tar 分发包中的驱动程序 mdriver.c 用于测试你的 mm.c 软件包的正确性、空间利用率和吞吐量。驱动程序由 malloclab-handout.tar 分发包中包含的一组跟踪文件控制。每个跟踪文件包含一系列分配、重新分配和释放指令，这些指令指导驱动程序按一定顺序调用你的 mm malloc、mm realloc 和 mm free 程序。驱动程序和跟踪文件将是我们用来评分你的提交的 mm.c 文件时使用的相同文件。

驱动程序 mdriver.c 接受以下命令行参数:

- -t： 在 tracedir 目录中查找默认的跟踪文件，而不是 config.h 中定义的默认目录。
- -f： 使用特定的 tracefile 进行测试，而不是默认的跟踪文件集。
- -h: 打印命令行参数的摘要。
- -l: 运行并测量 libc malloc 以及学生的 malloc 包。
- -v: 详细输出。以紧凑表格形式打印每个跟踪文件的性能分解。
- -V: 更详细的输出。在处理每个跟踪文件时打印额外的诊断信息。在调试时，用于确定哪个跟踪文件导致您的 malloc 软件包失败。

8 编程规则

- 你不应该更改 mm.c 中的任何接口。
- 你不应该调用任何与内存管理相关的库调用或系统调用。这排除了在代码中使用 malloc、calloc、free、realloc、sbrk、brk 或这些调用的任何变体的使用。
- 你不允许在 mm.c 程序中定义任何全局或静态的复合数据结构，例如数组、结构体、树或列表。但是，你允许在 mm.c 中声明全局标量变量，例如整数、浮点数和指针。

· 为了与返回 8 字节边界对齐块的 libc malloc 包保持一致，你的分配器必须始终返回 8 字节边界对齐的指针。驱动程序将为你执行此要求。

9 评估

如果你违反任何规则或代码存在错误导致驱动程序崩溃，你将不得分。否则，你的成绩将按以下方式计算：

- 正确性（20 分）。如果你的解决方案通过驱动程序程序执行的正确性测试，你将获得全部分数。对于每个正确的跟踪，你将获得部分分数。
- 性能（35 分）。将使用两个性能指标来评估你的解决方案：
  - 空间利用率：驱动程序使用的总内存量（即通过 mm malloc 或 mm realloc 分配但尚未通过 mm free 释放）与你的分配器使用的堆大小的峰值比率。最优比率等于 1。你应该找到良好的策略来最小化碎片，以使这个比率尽可能接近最优。
  - 吞吐量：每秒完成的平均操作数。

  驱动程序通过计算性能指数 P 来总结你的分配器的性能，P 是空间利用率和吞吐量的加权总和

$$P = wU + (1 - w)\min\left(1, \frac{T}{T}\right)$$

  其中 U 是你的空间利用率，T 是你的吞吐量，T 是默认跟踪下你的系统上 libc malloc 的估计吞吐量。性能指数更倾向于空间利用率而非吞吐量，默认 w = 0.6。

  观察到内存和 CPU 周期都是昂贵的系统资源，我们采用这个公式来鼓励内存利用率和吞吐量的均衡优化。理想情况下，性能指标将达到 P = w + (1 − w) = 1 或 100%。由于每个指标分别最多对性能指标贡献 w 和 1 − w，因此你不应该极端地只优化内存利用率或吞吐量。为了获得好成绩，你必须在这两者之间取得平衡。

- 风格（10 分）。
  - 你的代码应该分解为函数，并尽可能少地使用全局变量。
  - 你的代码应该以一个描述你的空闲和分配块的结构、空闲列表的组织以及你的分配器如何操作空闲列表的头部注释开始。每个函数都应该有头部注释，描述该函数的作用。

[1]T 的值是讲师在配置程序时在驱动程序中设定的一个常数（600 Kops/s）。

– 每个子程序都应该有一个头部注释，描述它做什么以及它是如何做的。
– 你的堆一致性检查器 mm check 应该全面且文档齐全。

如果你有一个好的堆一致性检查器以及良好的程序结构和注释，你将各获得 5 分。

10 提交说明

特定站点说明：在这里插入一段解释学生应如何提交他们的解决方案 mm.c 文件的段落。

11 提示

- 使用 mdriver -f 选项。在初始开发阶段，使用微小的跟踪文件将简化调试和测试。我们包含了两个这样的跟踪文件（short1,2-bal.rep），你可以使用它们进行初始调试。

- 使用 mdriver -v 和 -V 选项。-v 选项将为你提供每个跟踪文件的详细摘要。-V 选项还将指示每次读取每个跟踪文件的时间，这将帮助你隔离错误。

- 使用 gcc -g 编译并使用调试器。调试器将帮助你隔离和识别越界内存引用。

- 理解教科书中的 malloc 实现的每一行。教科书有一个基于隐式空闲列表的简单分配器的详细示例。以此作为起点。在完全理解简单隐式列表分配器的所有内容之前，不要开始工作你的分配器。

- 将指针算术封装在 C 预处理器宏中。内存管理中的指针算术由于需要大量类型转换而令人困惑且容易出错。通过为指针操作编写宏，你可以显著降低复杂性。请参考文本中的示例。

- 分阶段进行实现。前 9 个跟踪记录包含对 malloc 和 free 的请求。最后 2 个跟踪记录包含对 realloc、malloc 和 free 的请求。我们建议你首先确保你的 malloc 和 free 例程在前 9 个跟踪记录中能够正确且高效地运行。只有到那时，你才应该将注意力转向 realloc 的实现。初始阶段，可以在现有的 malloc 和 free 实现的基础上构建 realloc。但要获得真正好的性能，你需要构建一个独立的 realloc。

- 使用分析器。你可能会发现 gprof 工具有助于优化性能。

- 尽早开始！用几页代码就可以写出一个高效的 malloc 包。然而，我们可以保证这是你职业生涯到目前为止最困难和最复杂的代码之一。所以尽早开始，祝你好运！